FASTAPI

It is a modern Python web framework used to build API's quickly & efficiently.

It is commonly used for:

- REST API's
- Backend services
- Microservices
- ML model API's
- Database-backed Applications

Main Advantages

- High Performance
- Automatic request development
- Type-hint based development
- Automatic API documentation
- Supports asychronous programming
- Less boilerplate code

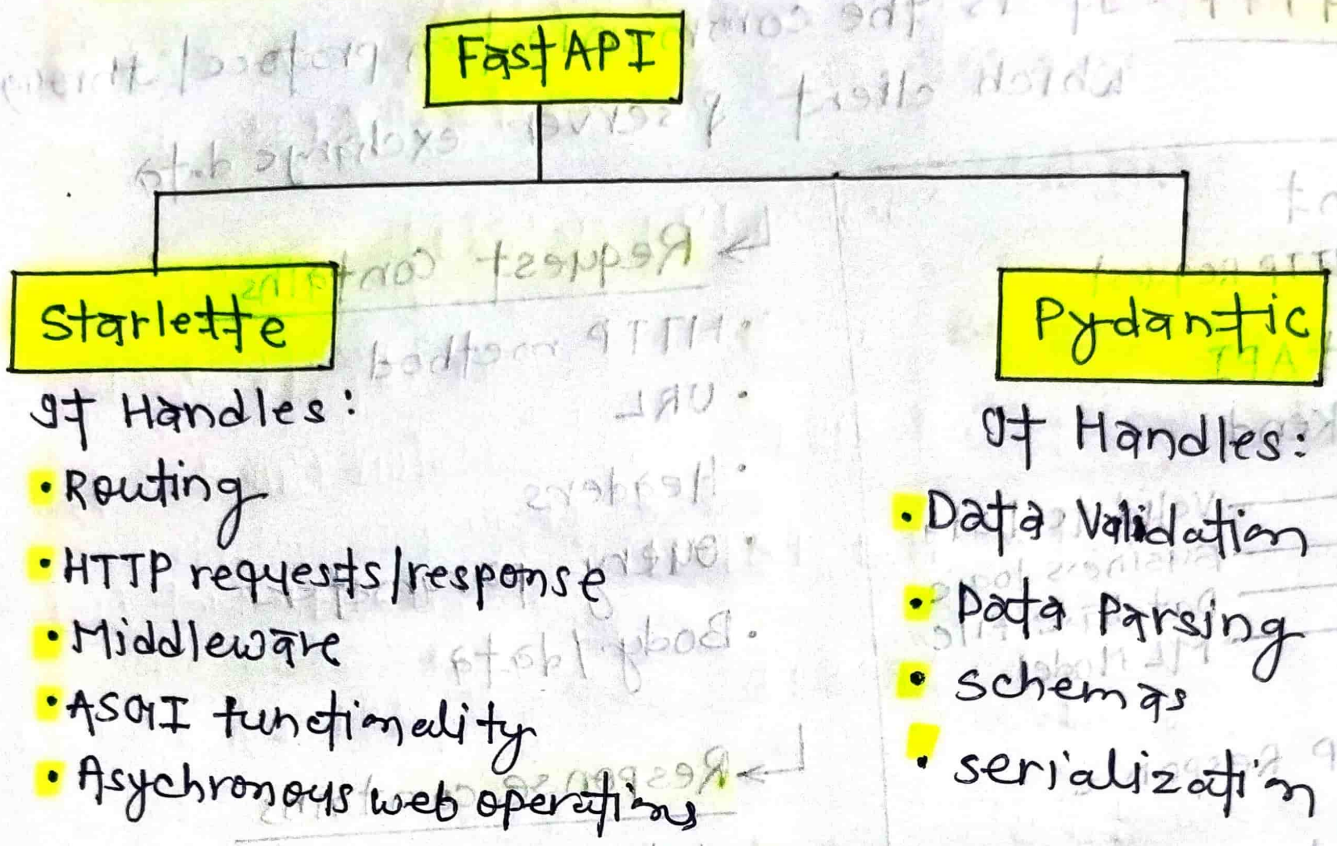
Installation

```
python -m venv venv
```

Install FastAPI & Uvicorn:

```
pip install fastapi uvicorn
```

→ FastAPI Architecture



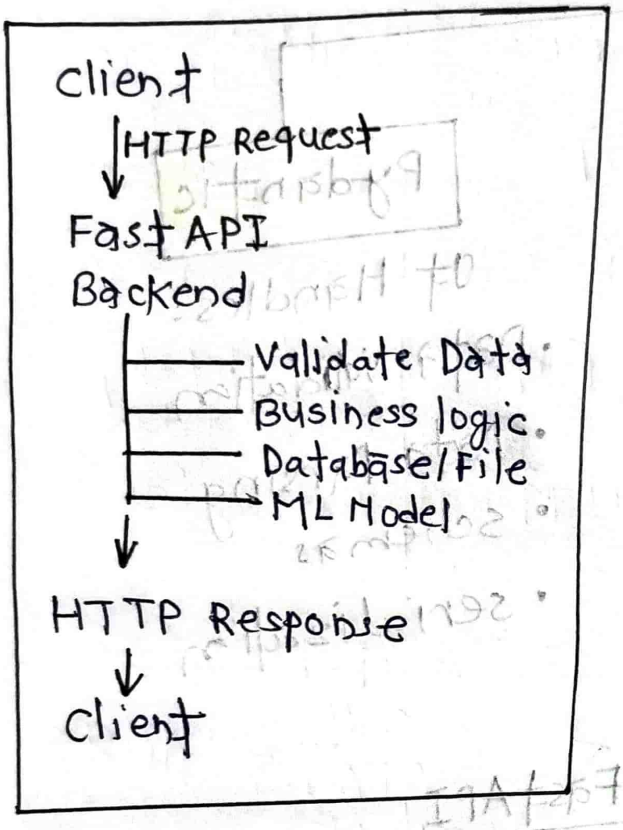
→ Basic Implementation of FastAPI

```
from fastapi import FastAPI
app = FastAPI() #create the FastAPI Application
@app.get("/")
# create a GET endpoint for the root URL "/"
def root():
# Function that runs when "/" is requested
    return {"message":"Hello FastAPI"}
    # Return response as JSON
```

Here: This code Imports FastAPI Backend/API Where we can define URL's (endpoints) that receive requests & return data to applications, websites or ML Models.

→ HTTP Protocol & Request-Response cycle

↳ **HTTP**: It is the communication protocol through which client & server exchange data



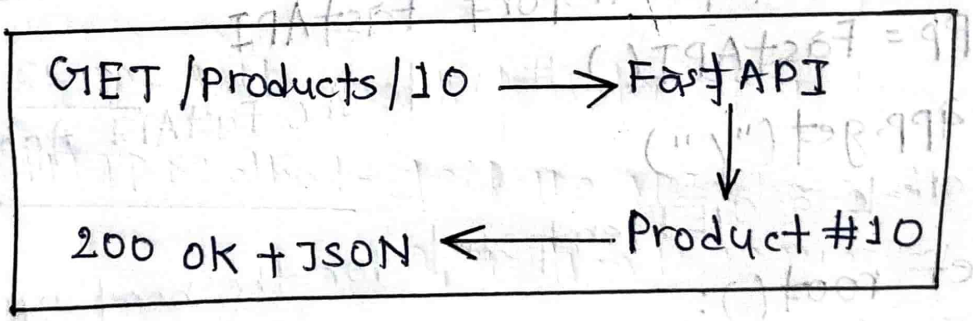
↳ **Request contains**

- HTTP method
- URL
- Headers
- Query parameters
- Body/data

↳ **Response contains**

- status code
- Headers
- Response body

- **Example**



→ HTTP Methods & CRUD

↳ **HTTP Methods**: These are standardized command used by a client to tell server what action it wants to perform on a resource.

HTTP Method	Meaning	Work
(1) GET	Retrieve	Gets data from the server
(2) POST	Submit / create	sends data to create a new resource
(3) PUT	Replace	Replaces the complete existing resource
(4) PATCH	Modify	Updates only specific fields
(5) DELETE	Remove	Delete a resource

• Example:

```
GET    /products
POST   /products
PUT    /products/101
PATCH  /products/101
DELETE /products/101
```

Here; /Produts - are the resource/ endpoint.

↳ **CRUD (Create, Read, Update, Delete)**

It represent the four fundamental operations we perform on stored data.

CRUD	Meaning	Explaination
C - Create	Add	create new data
R - Read	Retrieve	Get existing data
U - Update	Modify	change existing data
D - Delete	Remove	Delete existing data

↳ Correlation Betn HTTP Methods & CRUD

"HTTP Methods" are commonly used to Implement "CRUD" operation in a REST API:

CRUD operation	Common HTT Method	Example
Create	POST	POST/products
Read	GET	GET/products/101
Update	PUT/PATCH	PUT/products/101
Delete	DELETE	DELETE/products/101

CRUD → Defines the DATA operation → HTTP Method → Defines how the API request performs that operation

In short: CRUD - tells us what data operation we want
HTTP Method - tell the API how that operation is requested

↳ PUT vs PATCH

- PUT - Replace/update the complete resource
- PATCH - Update only selected fields

→ Static Vs Dynamic Routes

↳ **static Route**: It has a fixed URL path that does not change.

- **Example**

```
@app.get("/products")
def products():
    return {"message": "products"}
```

- **URL:** /products

every request goes to the same endpoint.

↳ **Dynamic Routes**

It contains a variable part of the URL called a path parameter, allowing the endpoint to handle different values.

or It is route where part of the URL can change to accept different values.

- **Example**

```
@app.get("/products/{product_id}")
def product(product_id: int):
    return {"id": product_id}
```

Here: {product_id} changes for each Product.

→ **Path Parameter**: It is a value placed directly inside the URL to identify or specify a particular resource.

- **Example:**

/products/50 Here, 50 is the path parameter

- **Example code -**

```
@app.get["/products/{product_id}")
def get_product(product_id: int):
    return {"product_id": product_id}
```

→ HTTP Exception

It is used to intentionally stop a request & return an HTTP error to the client.

↳ Use of it

Use it when something goes wrong, such as:

- Product doesn't exist
- User isn't authorized
- Invalid request
- Access is forbidden

↳ Example

```
from fastapi import HTTPException
@app.get("/products/{product_id}")
def get_product(product_id: int):
    if product_id < 0:
        raise HTTPException(
            status_code = 404,
            detail = "Product not
                      found")
    return {"product_id":
            product_id}
```

↳ common errors

400 → Bad request
401 → Unauthorized
403 → Forbidden
404 → Not Found
422 → Validation Error
500 → Internal server error

→ HTTP status code

It is a number sent by the server to tell the client the result of a request.

In short: tells whether the request succeeded or failed, & why.

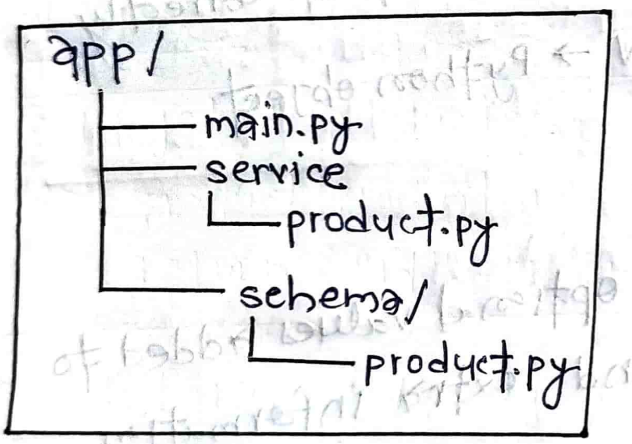
↳ classes

- 1xx → Informational
- 2xx → Sucess
- 3xx → Redirection
- 4xx → client error
- 5xx → server error

→ Service Layer

This layer keeps the application / business logic seprate from API routes.

↳ Examples



→ pathlib.path

"Path" is used to create & manage file paths easily & reliably.

↳ use :-

- cleaner than manually writing file paths
- Works across different operating system
- Makes path operations easier

↳ Ex

```
from pathlib import Path
DATA_FILE = (
    Path(__file__).parent.parent
    / "data"
    / "products.json")
```

→ Reading JSON

It means loading data from a JSON file into Python.

↳ Ex

```
import json
with open(DATA_FILE,"r",
encoding="utf-8") as f:
    products = json.load(f)
```

↳ Important code

- "r" → open file for reading
- encoding="utf-8" → Read UTF-8 text correctly
- json.load() → Convert JSON → Python object

→ Query parameters

these are optional values added to a URL after "?" to send extra information to the API.

↳ Example

```
/products?name=laptop
```

→ Query() Validation

" Query() " used to validate & set rules for query parameters.

↳ Ex

```
from fastapi import Query
@app.get("/products")
def products(name:str =Query(default=None, min_length=1,
                max_length=50)):
```

Common constraints

min_length → minimum text length
max_length → maximum text length

gt → greater than
ge → geather than or equal
lt → less than
le → less than or equal
min → minimum value
max → maximum value

- Ex

```
limit : int=Query (
        default = 10,
        ge = 1
        le = 100 )
```

it means $1 \leq limit \leq 100$

→ Search / Filtering

Filtering means selecting only the data that matches a user's search or condition.

↳ Ex

```
needle = name.strip().lower()
products = [p for p in products
           if needle in p.get("name", "").lower()]
```

↳ Flow

user input → strip() → lower() → compare → Filtered Results

→ Sorting

It means arranging data in a particular order, such as lowest-to-highest or highest-to-lowest

↳ **Example**

```
products = sorted(Products, key = lambda p: p.get("price", 0)
```

→ Pagination

It means dividing a large amount of data into smaller pages instead of returning everything at once.

↳ **Ex**

```
@app.get("/products")
def products(limit: int = 10, offset: int = 0):
    return products[offset : offset + Limit]
```

- limit - how many items to return
- offset - where to start

→ Path() Validation

Path() is used to validate values provided in URL paths.

↳ **Ex**

```
from fastapi import Path
@app.get("/products/{id}")
def product(id: str = Path(
        min_length = 1,
        max_length = 50)):
    return{"id" : id}
```

→ Pydantic BaseModel

BaseModel defines the expected structure, fields & data types of incoming data.

↳ Ex

```
from pydantic import BaseModel

class Product(BaseModel):
    name : str
    price : float
    stock : int
```

- In short - Pydantic model = data contract

→ Field()

It adds specific validation rules & constraints to pydantic fields

↳ Ex

```
from pydantic import BaseModel, Field
class Product (BaseModel):
    name: str = Field(min_length=2)
    price : float = Field (gt=0)
```

→ Useful Pydantic Types

Pydantic Types provide more specific validation than basic Python types.

↳ Ex

```
from typing import Literal
from uuid import UUID
from datetime import datetime
currency : Literal["INR"]
product_id : UUID
created_at : datetime
```

Common types:

```
Optional[str] → string or None
List[str] → list of strings
UUID → UUID value
datetime → date/time
AnyUrl → valid URL
Literal → specific allowed values
```

@field_validator

It is used for custom validation of one specific field.

Ex

```
from pydantic import BaseModel, field_validator
class Product(BaseModel):
    sku : str
    @field_validator ("sku")
    @classmethod
    def check_sku(cls, value):
        if "-" not in value:
            raise ValueError("Invalid SKU")
        return value
```

@model_validator

It is used when validation depends on multiple fields together.

- field_validator - one field
- model_validator - multiple fields

↳ Ex -

```
from pydantic import BaseModel, model_validator
class Product(BaseModel):
    stock : int
    is_active : bool
    @model_validator(mode = "after")
    def check(self):
        if self.stock == 0 and self.is_active:
            raise ValueError("Invalid Product")
        return self
```

→ @computed_field

It creates a calculated value from existing model fields.

↳ Ex

```
from pydantic import BaseModel, computed_yield
class Product(BaseModel):
    price : float
    discount : float
    @computed_field
    @property
    def final_price(self):
        return self.price * (1 - self.discount/100)
```

→ Nested Pydantic Models

A nested model is when one Pydantic model is used inside another model.

↳ Ex

```
class Seller(BaseModel)
    name:str
    email : str
class Product(BaseModel)
    name :str
    seller : Seller
```

→ **model_dump()**

It converts a Pydantic model into a Python dictionary.

↳ Ex

```
data = product.model_dump()
```

for JSON-compatible data

Ex-

```
data = product.model_dump(model="json")
```

→ **exclude_unset = True**

Used mainly with "PATCH" to keep only the fields actually provided by the user.

↳ Ex -

```
updates = product.model_dump(
    model="json"
    exclude_unset=True)
```

In short - It update only provided fields

→ **File-Based CRUD**

It means storing & managing application data using a file such as JSON instead of a database

- FastAPI + JSON file = simple file based CRUD system

→ **.POP()**

It removes an item from a Python list using its index.

↳ **Ex:**

```
products = ["Laptop", "Phone"]
products.pop(0)
```

→ **EmailStr**

It validates whether a value has a valid email format.

↳ **Use:** When our API accepts email addresses & we want automatic email-format validation.

↳ **Ex**

```
from pydantic import BaseModel, EmailStr
class Seller(BaseModel):
    email : EmailStr
```

→ **AnyUrl**

It validates whether value is a valid URL.

↳ **Use:** When our API accepts website links, profile URL's, Image URL's or other URL's.

↳ **Ex:**

```
from pydantic import BaseModel, AnyUrl
class Product(BaseModel):
    website : AnyUrl
```

→ **field_validator - mode = "after"**

" mode = "after" " runs our custom validator after Pydantic has performed its normal Validator/conversion.

↳ **Use:** When we want to apply our own validation rule after Pydantic has already checked the field.

↳ **Ex:**

```
@field_validator("sku", mode = "after")
@classmethod
def check_sku(cls, value):
    if "_" not in value:
        raise ValueError("Invalid SKU")
    return value
```

→ **Email Domain Validation**

custom validation can restrict emails to specific allowed domains.

↳ **Use:** when we want only email from certain organization or domains should be accepted.

↳ **Ex:**

```
@field_validator("email")
@classmethod
def check_email(cls, value):
    if not
value.endwith("@example.com")
    raise ValueError("Invalid domain")
    return value
```

→ **json.dump()**

It write a Python object into json file

↳ **Use** : use it when our need is to save or update API data in a JSON file.

- json.load() - JSON → Python
- json.dump() - Python → JSON

↳ **Ex**

```
with open("product.json", "w") as f:
    json.dump(products, f, indent=4)
```

→ **enumerate()**

It gives both the index & value while looping through a list.

↳ **Use**: when we need the position/index of an item while processing a list, such as finding an item to delete or update.

↳ **Ex**:

```
for index, product in
enumerate(products):
    if product["id"] == product_id:
        product.pop(index)
```

→ Dedicated Update Schema

update schema containing optimal fields so user can update only what they want.

↳ **Use**: use it mainly for PUT/PATCH endpoints to control which fields can be updated.

↳ **Ex**:

```
class ProductUpdate(BaseModel):
    name: str | None = None
    price: float | None = None
    stock: int | None = None
```

Ex PATCH -

```
{
  "price": 50000
}
```

→ Nested Updates

It can also be updated as part of a larger resource.

↳ **Use**: when a resource contains structured/nested data that also needs to be updated.

↳ **Ex**:

```
class ProductUpdate(BaseModel):
    seller: seller | None = None
    dimensions: Dimensions | None = None
```

→ **.any() for Duplicate checking**

"any()" returns True when at least one item satisfies a condn.

↳ **Use** - Use it to quickly check whether a duplicate value already exists.

↳ **Ex** -

```
if any(p.get("sku") == product.get("sku")
for p in products:
    raise ValueError("SKU already exists")
```

→ **save_product() as a single Persistence Function**

A persistence helper centralizes the logic for writing product data to storage.

↳ **Use** - we use it so POST, PUT/PATCH & DELETE can all save data through the same function instead of repeating file-writing code.

↳ **Ex** -

```
def save_product(products):
    with open(DATA_FILE, "w",
    encoding="utf-8") as f:
        json.dump(products, f,
    indent=4, ensure_ascii=False)
```

* indent=4 → makes JSON readable
* ensure_ascii=False → preserve characters like ê, ₹, 中文 etc.

→ **201 Created** - HTTP 201 means a new resource was sucessfully created.

Use it as the response status for sucessful POST /create operation

Server-Side Metadata

It is information generated by the backend rather than supplied by the client. Use it for values such as unique ID's & creation timestamps.

↳ **Ex:**

```
product_dict["id"] = str(uuid4())
product_dict["created_at"] =
datetime.now(timezone.utc).isoformat()
```

↳ **Idea:** client sends product data → server adds system information ↓ saves product

↳ **uuid4()** : It generates a random UUID used as a unique Identifier.

Use it when resources need automatically generated unique ID's.

↳ **datetime.now(timezone.utc)**

Gets the current date & time in UTC with timezone information.

Use it when storing reliable timestamps for created / updated resources.

* **isoformat()** = standard text representation of date/time.

→ Error Translation : ValueError → HTTPException

Error translation converts an internal Python exception into an appropriate HTTP error for API client.

↳ **Use:** use it to keep service-layer logic independent from HTTP while routes convert errors into HTTP response.

↳ **Ex:**

```
try:
    return add_product(product)
except ValueError as e:
    raise
HTTPException(status_code=400,
detail=str(e))
```

→ response_model as output Filtering

"response_model" can remove fields from the returned data that are not included in the response schema.

↳ **Use:** use it when our internal data contains field that should not be exposed through the API

↳ **Ex:**

```
@app.get("/products", response_model=
                ProductResponse
def get_product():
    return product
```

→ response_model for API Documentation

FastAPI uses response_model to describe the expected response structure in OpenAI documentation

↳ **Use**: it's used to make our /docs documentation clearly show what the API returns.

↳ **Ex**

```
@app.get("/products",
response_model=List[Product])
def get_products():
    ...
```

→ Generic Collection Response Models

A response model can describe a collection containing a specific type of object.

* List[Product] = list containing product object

→ Primitive Response Models

"response_model" can also describe simple return types such as str, int or dict.

↳ **Use**: when endpoint returns a simple value rather than a pydantic object.

↳ **Ex**:

```
@app.get("/message", response_model=str)
def message():
    return "Hello"
```

→ Pydantic schema as Response Model

A pydantic model can define exactly which fields & structure an API response should have.

↳ **Use:** when we want a structured & controlled API response.

↳ **Ex:**

```
class ProductResponse(BaseModel):
    name: str
    price : float
@app.get("/product", response_model=Product
                                    Response
def get_product():
    ...
```

→ JSONResponse - Dynamic Response control

"JSONResponse" lets we explicitly construct an HTTP JSON response instead of relying only on FastAPI's automatic response handling.

↳ **Use:** when the status code or response content needs to be decided dynamically

↳ **Ex:**

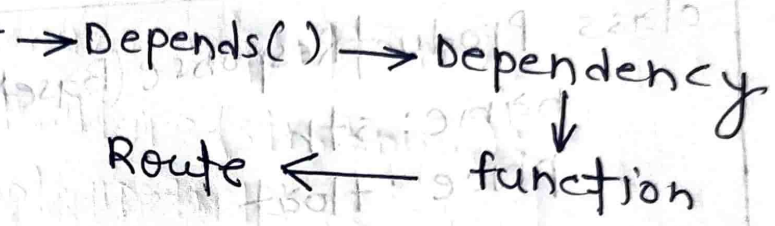
```
return JSONResponse(status_code=200,
        content={"success": True})
```

→ Dependency Injection - Depends()

It allows FastAPI to automatically run a shared function & provide its result to an endpoint.

↳ **Use:** used it for reusable logic such as authentication, loading data, logging or common preprocessing.

↳ **Flow:**



↳ **Ex**

```
from fastapi import Depends
def common_logic():
    return {"status": "validated"}
@app.get("/items")
def get_items(dep: dict = Depends(
common_logic)):
    return dep
```

→ Middleware - Global Request / Response Processing

Middleware runs around every HTTP request & response in the application.

↳ **Use -** for global tasks such as logging, timing requests, authentication-related processing, or modifying responses.

↳ Ex:

```
@app.middleware("http")
async def custom_lifecycle(request, call_next):
    print("Before request")
    response = await
call_next(request)
    print("After request")
    return response
```

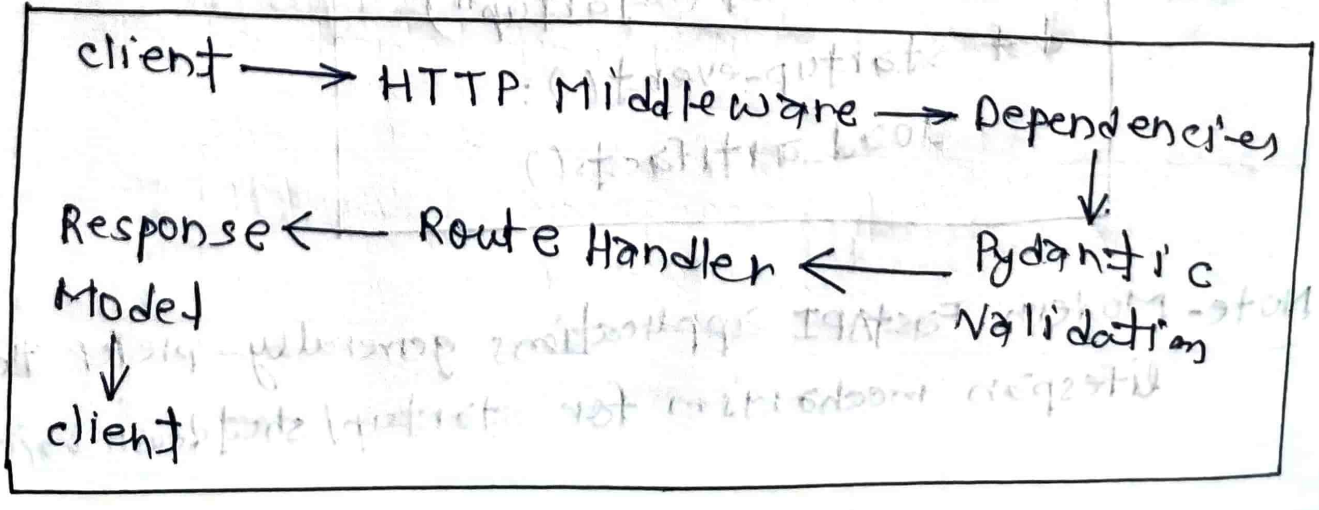
↳ Important:

- Before call_next() → incoming request processing
- call_next() → passes request to the route
- After call_next() → outgoing response processing.

→ Request Processing Lifecycle

The request lifecycle is the order in which FastAPI processes an incoming request.

↳ Use: Use this mental model to understand where middleware, dependencies, validation, routes & response fit together.



→ Enum for Restricted Input choices

"Enum" restricts a field to a predefined set of allowed values.

↳ **Use:** When an API field should accept only specific choices, such as fuel type, transmission, role or status.

↳ **Ex:**

```
from enum import Enum
class FuelType (str, Enum):
    petrol: "petrol"
    diesel: "Diesel"
    cng = "CNG"
```

→ Startup Model Loading

It means loading the ML model when the FastAPI application starts.

↳ **Use:** When load heavy model once & keep it in memory instead of loading it from disk for every prediction request.

↳ **Ex-**

```
@app.on_event ("startup")
def startup-event():
    load_artifacts()
```

Note- Modern FastAPI applications generally prefer the lifespan mechanism for startup/shutdown logic.

→ **request.post()**

It sends an HTTP POST request from the streamlit frontend to the FastAPI backend.

↳ **Use:** it to send user-entered data to an API end-point.

↳ **Ex-**

```
response = requests.post("http://localhost:8000/predict", json=data)
```

→ **0.0.0.0**

It tells the server to listen all available network interfaces.

↳ **Use** - it in cloud/container deployments so the hosted application can recieve external request.

↳ **Ex** -

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

→ **$PORT**

It is an enviroment variable containing the port assigned by the hosting platform

↳ **use** - it when deploying FastAPI to platforms that dynamically provide the application's listening port.

↳ **ex** -

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

→ CORS - cross origin Requests

CORS controls whether a frontend from one origin can make requests to a backend on another origin.

↳ **Use:** when our frontend & FastAPI backend are hosted on different domains/ports.

↳ **Ex:**

```
from fastapi.middleware.cors import
CORSMiddleware

app.add_middleware(CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],)
```

↳ **Example**

```
streamlit → http://frontend.com
FastAPI → http://api.com
```

→ runtime.txt

It specifies the Python runtime version required by certain deployment platforms.

↳ **Use:** when the deployment platform supports runtime.txt & we need to specify a particular Python version.

→ **Build command**

It prepares the deployment environment while a start command launches the application.

↳ **Use**: the correct commands when configuring a cloud service.

↳ **Build Code Ex**

```
pip install -r requirements.txt
```

* **Build Command** = prepare Application
* **start command** = run Application

PROJECT-1
CAR PRICE PREDICTION USING ML & FASTAPI

→ **Overview**

- Build a full-stack ML application to predict second-hand car selling prices
- Trained ML model is saved as a .pkl artifact for prediction
- FastAPI serves as the backend REST API.
- Pydantic validates car input data & restricts categorical values using Enum.
- ML model is loaded at Application startup & reused for Prediction.
- Created a "POST /predict" endpoint for real-time Price Prediction.
- streamlit provides the frontend UI for entering car details.
- streamlit sends user data to FastAPI using HTTP POST requests.
- FastAPI validates the data → runs the ML Model → returns the predicted price.
- Added CORS to support to support frontend-backend communication across different origins.
- Backend can be deployed on Render & frontend on streamlit community cloud
- Overall architecture: streamlit → FastAPI → ML Model → Prediction → streamlit

→ **Main.py**

```
from dotenv import load_dotenv
import os
from fastapi import FastAPI, HTTPException, Query,
Path, Depends, Request
from fastapi.responses import JSONResponse
from service.products import (
        get_all_products,
        add_product,                    #service layer functions
        remove_product,
        change_product,
        load_product )
from schema.product import Product, ProductUpdate
from uuid import uuid4, UUID
from datetime import datetime
from typing import List, Dict
load_dotenv()
app = FastAPI()
==========
#DEPENDENCY EXAMPLE
==========
def common_logic():          #common function that can be
    return "Hello there"     injected using Depends()
==========
#ROOT ENDPOINT
==========
@app.get("/", response_model = dict)
def root(dep=Depends(common_logic)):      #Read BASE_URL
    DB_PATH = os.getenv("BASE_URL")       from environment
    return JSONResponse(status_code=200,  variable
    content={"message":"Welcome to FastAPI", "dependency": dep,
    "data_path": DB_PATH,})
```

GET ALL PRODUCTS, SEARCH + SORTING + PAGINATION

```
@app.get("/products", response_model=Dict)
def list_products(
    dep=Depends(load_products),
    #Dependency load products from storage
    name: str = Query(
        default=None,
        min_length=1,
        max_length=50,
        description="search by product name
(case insensitive)", example="Energy 3Pcs",),
    #optional search Parameter
        sort_by_price: bool = Query(default=False,
            description="sort products by price",),
        #Enable price sorting
        order: str = Query(
            default="asc",
            description="sort order when
        sort_by_price=true(asc, desc)",
        #sorting order
        limit: int = Query(
            default=10,
            ge=1,
            le=100,
            description="Number of items to
        return",),
    #No. of products returned
```

```
    offset: int = Query(
        default=0,
        ge=0
        description="Pagination offset",),):
    #No. of products to skip
    products = dep
    #get products returned by dependency
    if name:
        needle = name.strip().lower()
        products = [p for p in products if
needle in p.get("name","").lower()
    #search product by name
    if not products:
        raise HTTPException(status_code=404,
                detail=f"No product found matching
name={name}"
    #Return error if no products exist/match
    if sort_by_price:
        reverse = order == "desc"
        products = sorted(products, key=lambda p:
p.get("price",0), reverse=reverse,)
    #sort product by price
    total = len(products) # count products before pagination
    products = products[offset:offset+limit]
    #Apply pagination
    return{"total": total,
           "limit": limit,
           "items": products,}
    #Return final response
```

```
#GET PRODUCT BY ID
@app.get("/products/{product_id}",
response_model=Dict)
def get_product_by_id(
    product_id:str = Path(
        min_length=36,
        max_length=36,
        description="UUID of the Products",
    #validate product ID length
example="c47ea2457-c489-4bfg-9dd5-b464r0ebe343"
        )):
    products=get_all_products()
    #Load all products
    for products in products:
        if product["id"]==product_id:
            return product
    #Product was not found
    raise HTTPException(status_code=404,
        detail="product not found!",)
#CREATE PRODUCT
@app.post("/products",status_code=201)
def create_product(product: Product)
    product_dict=product.model_dump(mode="json")
    #convert Pydantic model to dictionary
    product_dict["id"]=str(uuid4())
    #Generate unique product ID
    product_dict["created_at"]=(datetime.utcnow().
        isoformat()+"Z")
```

```
        try:
            add_product(product_dict)
            #save product using service layer
        except ValueError as e:
            raise HTTPException(status_code=400,
              detail=str(e),)
            #convert service error into HTTP error
        return
    product.model_dump(mode="json")
    # Return created product
# = = =
```

UPDATE PRODUCT

```
@app.put("/products/{product_id}")
def update_product(
    product_id: UUID = Path(...,
        description="Product UUID",)
    #validate product UUID
    payload: ProductUpdate = ...,):
    try:
        update_product=change_product(
            str(product_id), payload.model_dump(
            mode="json", exclude_unset=True,),)
        #Convert only supplied field to dictionary
        return update_product
        #Return updated product
    except ValueError as e:
        raise HTTPException(status_code=404,
            detail=str(e),)
    #Product not found/invalid update
```